

Week - 8

Q1:

```
#include <stdio.h>
#include <cuda.h>

#define N 3

//One thread per row
__global__ void add_row(int *A, int *B, int *C) {
    int row = threadIdx.x;
    if (row < N) {
        for (int j = 0; j < N; j++) {
            C[row * N + j] = A[row * N + j] + B[row * N + j];
        }
    }
}

//Thread per column
__global__ void add_col(int *A, int *B, int *C) {
    int col = threadIdx.x;
    if (col < N) {
        for (int i = 0; i < N; i++) {
            C[i * N + col] = A[i * N + col] + B[i * N + col];
        }
    }
}

//One thread per element
__global__ void add_element(int *A, int *B, int *C) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    if (i < N && j < N) {
        C[i * N + j] = A[i * N + j] + B[i * N + j];
    }
}
```

```

void printMatrix(int *M) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", M[i * N + j]);
        }
        printf("\n");
    }
    printf("Adarsh Ranjan, 230962278");
    printf("\n");
}

int main() {
    int A[N*N] = {1,2,3,4,5,6,7,8,9};
    int B[N*N] = {9,8,7,6,5,4,3,2,1};
    int C[N*N];

    int *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, N*N*sizeof(int));
    cudaMalloc((void**)&d_B, N*N*sizeof(int));
    cudaMalloc((void**)&d_C, N*N*sizeof(int));

    cudaMemcpy(d_A, A, N*N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, N*N*sizeof(int), cudaMemcpyHostToDevice);

    printf("Matrix A:\n"); printMatrix(A);
    printf("Matrix B:\n"); printMatrix(B);

    printf("Addition (Row-wise):\n");
    add_row<<<1, N>>>(d_A, d_B, d_C);
    cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
    printMatrix(C);

    printf("Addition (Column-wise):\n");
    add_col<<<1, N>>>(d_A, d_B, d_C);
    cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
    printMatrix(C);
}

```

```

printf("Addition (Element-wise):\n");
dim3 threads(N, N);
add_element<<<1, threads>>>(d_A, d_B, d_C);
cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
printMatrix(C);

cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);

return 0;
}

```

```

• (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week8$ nvcc q1.cu -o q1
• (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week8$ ./q1
Matrix A:
1 2 3
4 5 6
7 8 9
Adarsh Ranjan, 230962278
Matrix B:
9 8 7
6 5 4
3 2 1
Adarsh Ranjan, 230962278
Addition (Row-wise):
10 10 10
10 10 10
10 10 10
Adarsh Ranjan, 230962278
Addition (Column-wise):
10 10 10
10 10 10
10 10 10
Adarsh Ranjan, 230962278
Addition (Element-wise):
10 10 10
10 10 10
10 10 10
Adarsh Ranjan, 230962278
• (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week8$

```

Q2:

```

#include <stdio.h>
#include <cuda.h>

```

```

#define N 3

//One thread per row
__global__ void mul_row(int *A, int *B, int *C) {
    int row = threadIdx.x;
    if (row < N) {
        for (int j = 0; j < N; j++) {
            int sum = 0;
            for (int k = 0; k < N; k++) {
                sum += A[row * N + k] * B[k * N + j];
            }
            C[row * N + j] = sum;
        }
    }
}

//One thread per column
__global__ void mul_col(int *A, int *B, int *C) {
    int col = threadIdx.x;
    if (col < N) {
        for (int i = 0; i < N; i++) {
            int sum = 0;
            for (int k = 0; k < N; k++) {
                sum += A[i * N + k] * B[k * N + col];
            }
            C[i * N + col] = sum;
        }
    }
}

//One thread per element
__global__ void mul_element(int *A, int *B, int *C) {
    int i = threadIdx.x;
    int j = threadIdx.y;
    if (i < N && j < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[i * N + k] * B[k * N + j];
        }
        C[i * N + j] = sum;
    }
}

```

```

    }

}

void printMatrix(int *M) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", M[i * N + j]);
        }
        printf("\n");
    }
    printf("Adarsh Ranjan, 230962278");
    printf("\n");
}

int main() {
    int A[N*N] = {1,2,3,4,5,6,7,8,9};
    int B[N*N] = {9,8,7,6,5,4,3,2,1};
    int C[N*N];

    int *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, N*N*sizeof(int));
    cudaMalloc((void**)&d_B, N*N*sizeof(int));
    cudaMalloc((void**)&d_C, N*N*sizeof(int));

    cudaMemcpy(d_A, A, N*N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, N*N*sizeof(int), cudaMemcpyHostToDevice);

    printf("Matrix A:\n"); printMatrix(A);
    printf("Matrix B:\n"); printMatrix(B);

    printf("Multiplication (Row-wise):\n");
    dim3 threads(N, N);
    mul_row<<<1, N>>>(d_A, d_B, d_C);

```

```

    cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
    printMatrix(C);

    printf("Multiplication (Column-wise):\n");
    mul_col<<<1, N>>>(d_A, d_B, d_C);
    cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
    printMatrix(C);

    printf("Multiplication (Element-wise):\n");
    mul_element<<<1, threads>>>(d_A, d_B, d_C);
    cudaMemcpy(C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
    printMatrix(C);

    printf("Adarsh Ranjan, 230962278");

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}

```

```

● (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week8$ nvcc q2.cu -o q2
● (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week8$ ./q2
Matrix A:
1 2 3
4 5 6
7 8 9
Adarsh Ranjan, 230962278
Matrix B:
9 8 7
6 5 4
3 2 1
Adarsh Ranjan, 230962278
Multiplication (Row-wise):
30 24 18
84 69 54
138 114 90
Adarsh Ranjan, 230962278
Multiplication (Column-wise):
30 24 18
84 69 54
138 114 90
Adarsh Ranjan, 230962278
Multiplication (Element-wise):
30 24 18
84 69 54
138 114 90
Adarsh Ranjan, 230962278

```